TEACHING WALLAH

Teaching Exam

Subject – Computer Science

Topic – Digital Logic

Lecture No.- 9

By- Rahul Sir

Topics to be Covered

Topic

1:- SOP-> K-map(2,3,4 Variable)
2:- POS-> K-map(2,3,4 Variable)
3:- K-Map using Don't CARE
4:- Introduction to Combinational Circuit
5:- Types of Combinational Circuit

Minimization Of Boolean Expressions-

There are following two methods of minimizing or reducing the boolean expressions-

Methods To Minimize Boolean Expressions

- By Using Laws of Boolean Algebra
- By Using Karnaugh Maps also called as K Maps

1:-By using laws of Boolean Algebra
2:-By using Karnaugh Maps also called as K Maps

1:- K-Map is graphical representation used for simplifying the Boolean expressions

2:- For a Boolean Consisting of n-variables, number of cells required in K Map = 2^n cells.

$\boxed{n=2}$, no of cell $= 2^2 = 4$

$= n$ $\quad = 2^n$

3:-K-Map is based on Grey Code (Unit distance code).

4:- K-Map is based on three type of input values {0 , 1, don't care(x) }

SOP $x = 1$

POS $x = 0$

Karnaugh Maps

- Another approach to simplification is called the Karnaugh map, or K-map.
- A K-map is a truth table graph, which aids in visually simplifying logic.
- It is useful for up to 5 or 6 variables, and is a good tool to help understand the process of logic simplification.
- The algebraic approach we have used previously is also used to analyze complex circuits in industry (computer analysis).
- At the right is a 2-variable K-map.
- This very simple K-map demonstrates that an n-variable K-map contains all the combination of the n variables in the K-map space.

	$\bar{y}$	y
$\bar{x}$	00 0	01 1
x	10 2	11 3

This minterm is expressed as $f = \bar{x}y$

Two-Variable K-map, labeled for SOP terms. Note the four squares represent all the combinations of the two K-map variables, or minterms, in x & y (example above).

$$Y = \bar{A}\bar{B} + A\bar{B} + \bar{A}B$$

$$= \bar{B}(\underbrace{A+\bar{A}}_{1}) + \bar{A}B$$

$$= \bar{B}\cdot 1 + \bar{A}B$$

$$= \bar{B} + \bar{A}\cdot B \longrightarrow \text{Distribution Law}$$

Or

Absorption Law

$$= (\bar{B}+\bar{A})\cdot(\underbrace{\bar{B}+B}_{1})$$

$$= (\bar{B}+\bar{A})\cdot 1$$

$$\boxed{Y = \bar{A}+\bar{B}}$$

① $A\cdot(B+C)$

$A\cdot B + A\cdot C$

② $A + B\cdot C$

$(A+B)\cdot(A+C)$

A	B	
0	0	$\to 0$
0	1	$\to 1$
1	1	$\to 3$
1	0	$\to 2$

10·

11

$2^1 2^0$

$$\text{SOP} \leftarrow Y = \overset{m}{\bar{A}\bar{B}} + \overset{m}{A\bar{B}} + \overset{m}{\bar{A}B}$$

$$\qquad\qquad 00 \qquad 10 \qquad 01$$

A(High) → A | B(High) → B

A(Low) → $\bar{A}$ | B(Low) → $\bar{B}$

$n = 2$

No of cell $= 2^n = 2^2 = 4$

A \ B	0	1
0	1 (0)	1 (1)
1	1 (2)	0 (3)

Grouping List

$A=0$, $B=0$, $Y=1$

$A=0$, $B=1$, $Y=1$

Grouping 2nd

$A=0$, $B=0$, $Y=1$

$A=1$, $B=0$, $Y=1$

$$\boxed{Y = \bar{A} + \bar{B}}$$

$$Y = \bar{A}\bar{B}C + \bar{A}B\bar{C} + \bar{A}\bar{B}\bar{C} + ABC$$

001   010   000   111 Ans

$n = 3$

no of cell $= 2^3 = 8$

$2^0 \quad 2^1 \quad 2^2 \quad 2^3 \quad 2^4$

1   2   4

Group

$A=0, B$

$A=0$

1 unit   1 unit   1 unit

C	00	01	11	10
	$-_{0}$	1_{1}	0_{3}	1_{2}
1	0_{4}	0_{5}	1_{7}	0_{6}

Single

$A=1, B=1, C=1, Y=1$

Side pair group

$A=0, B=0, C=0, Y=1$

$A=0, B=1, C=0, Y=1$

$$Y = \bar{A}\bar{B} + \bar{A}\bar{C} + ABC$$

$$Y = \sum(m_1, m_3, m_4, m_5, m_7)$$

A \ BC	00	01	11	10
0	0 $_0$	1 $_1$	1 $_3$	0 $_2$
1	1 $_4$	1 $_5$	1 $_7$	0 $_6$

Quad

A=0, B=0, C=1, Y=1
A=0, B=1, C=1, Y=1
A=1, B=0, C=1, Y=1
A=1, B=1, C=1, Y=1

Pair

A=1, B=0, C=0, Y=1
A=1, B=0, C=1, Y=1

$$Y = C + A\overline{B}$$

Three-Variable Karnaugh Map

- A useful K-map is one of three variables.
- Each square represents a 3-variable minterm or maxterm.
- All of the 8 possible 3-variable terms are represented on the K-map.
- When moving horizontally or vertically, <u>only 1 variable changes between adjacent squares</u>, never 2. <u>This property of the K-map, is unique and accounts for its unusual numbering system</u>.
- The K-map shown is one labeled for SOP terms. It could also be used for a POS problem, but we would have to re-label the variables.

	$\bar{y}\bar{z}$	$\bar{y}z$	yz	$y\bar{z}$
$\bar{x}$	000 0	001 1	011 3	010 2
x	100 4	101 5	111 7	110 6

As an example, this minterm cell (011) represents the minterm $f = \bar{x}yz$.

Four Variable Karnaugh Map

- A 4-variable K-map can simplify problems of four Boolean variables.*
- The K-map has one square for each possible minterm (16 in this case).
- Migrating one square horizontally or vertically never results in more than <u>one</u> variable changing (square designations also shown in hex).

* Note that on all K-maps, the left and right edges are a common edge, while the top and bottom edges are also the same edge. Thus, <u>the top and bottom rows are adjacent, as are the left and right columns</u>.

	$\bar{y}\bar{z}$	$\bar{y}z$	yz	$y\bar{z}$
$\bar{w}\bar{x}$	0000 0 0	0001 1 1	0011 3 3	0010 2 2
$\bar{w}x$	0100 4 4	0101 5 5	0111 7 7	0110 6 6
wx	1100 C 12	1101 D 13	1111 F 15	1110 E 14
$w\bar{x}$	1000 8 8	1001 9 9	1011 B 11	1010 A 10

Note that this is still an SOP K-map.

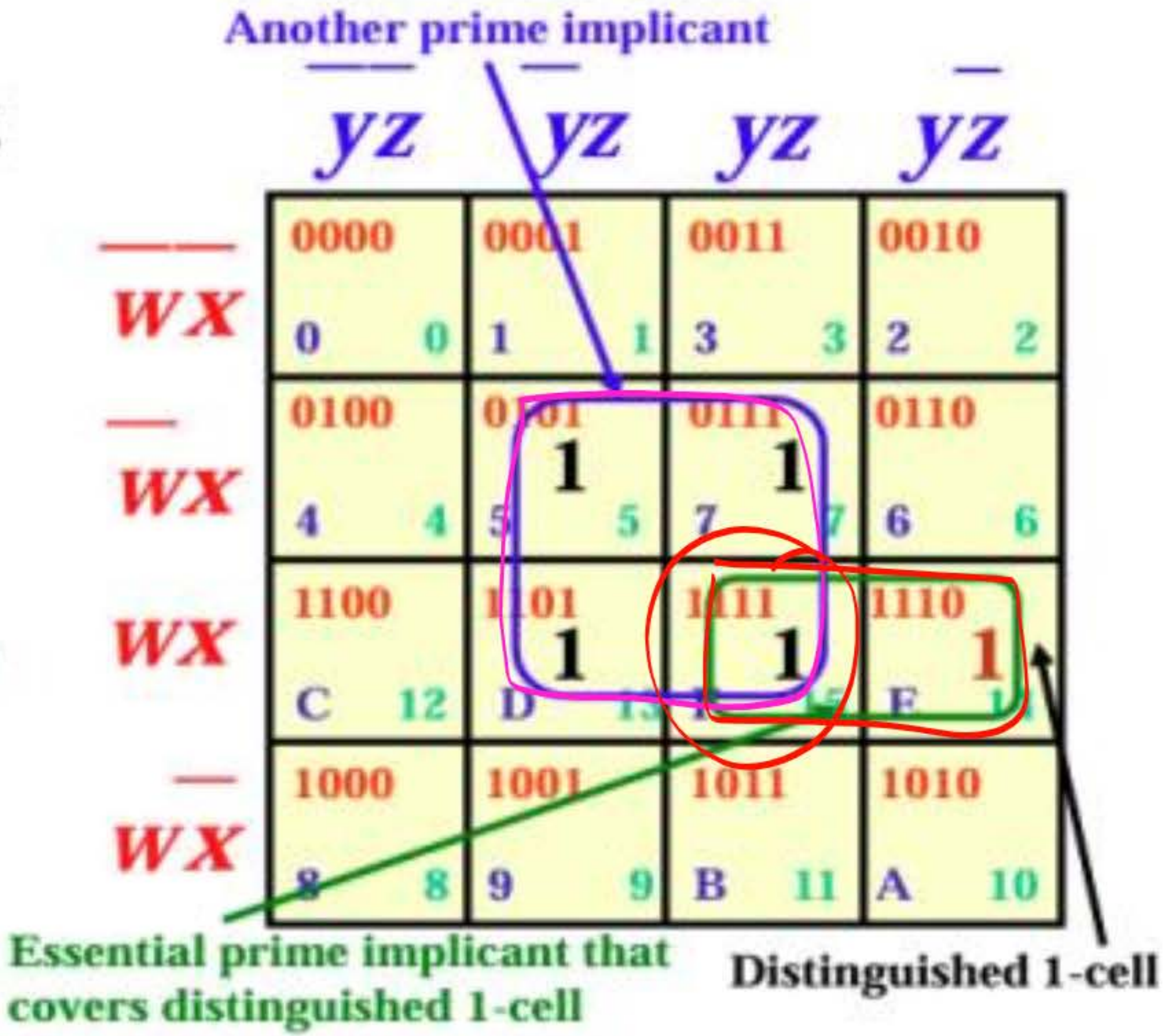
Prime Implicants

- We will be simplifying Boolean functions plotting their values on a K-map and grouping them into prime implicants.
- What is a prime implicant? It is an implicant that covers as many 1 values (SOP K-map) or 0 values (POS K-map) as possible, yet still retains the identity of implicant (# of cells = power of 2, rectangular or square shape).
- Some SOP prime implicants are shown on the adjoining K-map.

	$\overline{y}\,\overline{z}$	$\overline{y}z$	yz	$y\overline{z}$
$\overline{w}\,\overline{x}$	0000 1 0 0	0001 0 1 1	0011 1 3 3	0010 1 2 2
$\overline{w}x$	0100 1 4 4	0101 0 5 5	0111 1 7 7	0110 1 6 6
wx	1100 1 C 12	1101 1 D 13	1111 1 F 15	1110 1 E 14
$w\overline{x}$	1000 1 8 8	1001 0 9 9	1011 0 B 11	1010 0 A 10

More Karnaugh Map Terminology

- **Distinguished 1-cell**: A single minterm that can be covered by only one prime implicant.
- **Essential prime implicant**: A prime implicant that covers one or more distinguished 1-cells.
- Note: Every fully minimized Boolean expression must include all of the essential prime implicants of f.
- In the K-map at right, the Boolean minterm $f = wxy\bar{z}$ is a distinguished 1-cell, and the essential prime implicant $f = wxy$ is the only prime implicant that includes it.



"Prime Implicants"

- As noted two slides back, a "prime implicant" is the largest square or rectangular implicant of cells occupied by a 1 (SOP) or 0 (POS). Thus a prime implicant will have 1, 2, 4, or 8 cells (16 is a trivial prime).
- How do we determine the Boolean expression for a prime implicant?
- The Boolean expression for an SOP prime implicant is determined by creating a new minterm whose only variables are those that <u>do NOT change value (0→1 or 1→0) over the extent of the prime implicant</u>.
- Thus the prime implicant at right may be represented by the Boolean expression: $f = xz$.

Since x & z do not change value over the implicant, they are the variables in the new Boolean minterm.

	$\overline{y}\,\overline{z}$	$\overline{y}z$	yz	$y\overline{z}$
$\overline{w}\,\overline{x}$	0000 0 0	0001 1 1	0011 3 3	0010 2 2
$\overline{w}x$	0100 4 4	0101 1 5 5	0111 1 7 7	0110 6 6
wx	1100 C 12	1101 1 D 13	1111 1 F 15	1110 E 14
$w\overline{x}$	1000 8 8	1001 9 9	1011 B 11	1010 A 10

<u>Example of a prime implicant</u>

Logic Simplification – an SOP Example

- We simplify a Boolean expression by finding its prime implicants on a K-Map.
- To do this, populate the K-map as follows:
  - For an SOP expression,* find the K-Map cell for each minterm of function f, and place a one (1) in it. Ignore 0's.
  - Circle groups of cells that contain 1's.
  - The number of cells enclosed in a circle must be a power of 2 and square or rectangular.
  - It is okay for groups of cells to overlap.
  - Each circled group of cells corresponds to a prime implicant of f.
  - Note that the more cells a given circle encloses, the fewer variables needed to specify the implicant!

	$\overline{yz}$	$\bar{y}z$	yz	$y\bar{z}$
$\overline{wx}$	0000 0 0	0001 1 1	0011 3 3	0010 2 2
$\bar{w}x$	0100 1 4 4	0101 1 5 5	0111 1 7 7	0110 1 6 6
wx	1100 C 12	1101 1 D 13	1111 1 F 15	1110 E 14
$w\bar{x}$	1000 8 8	1001 9 9	1011 B 11	1010 A 10

* A POS example will follow.

Using the K Map for Logic Simplification (2)

- Another example:
  - The implicants of f are 4, 6, 12, 14.
  - This corresponds to the function:
    $f = \overline{w}x\overline{y}\overline{z} + \overline{w}xy\overline{z} + wx\overline{y}\overline{z} + wxy\overline{z}$
  - We create a <u>prime implicant</u> by grouping the 4 cells representing f.
  - On the wx axis, the cells are 1 whether or not w is one, and always when x is 1 (w not needed).
  - On the yz axis, the cells are 1 whether or not y is one, and always when z is 0 (y not needed).
  - Thus the simplified expression for f is: $f = x\overline{z}$.

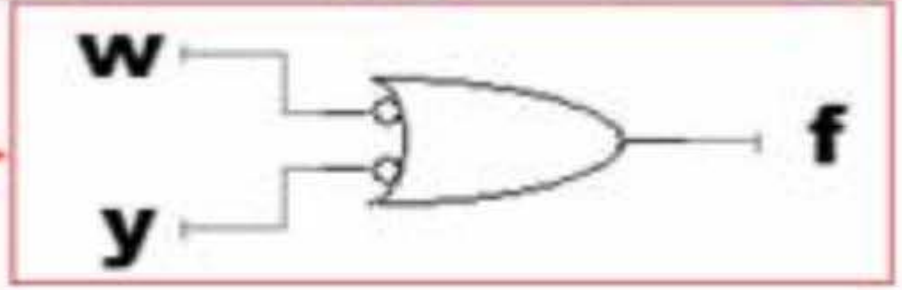
	$\overline{y}\,\overline{z}$	$\overline{y}z$	yz	$y\overline{z}$
$\overline{w}\,\overline{x}$	0000 0 0	0001 1 1	0011 3 3	0010 2 2
$\overline{w}x$	0100 1 4 4	0101 5 5	0111 7 7	0110 1 6 6
wx	1100 1 C 12	1101 D 13	1111 F 15	1110 1 E 14
$w\overline{x}$	1000 8 8	1001 9 9	1011 B 11	1010 A 10

Remember: For purposes of grouping implicants, <u>the top and bottom rows of the K-map are considered adjacent, as are the right and left columns</u>.
This grouping takes advantage of the fact that the left and right columns are adjacent.

A POS K-Map

- On a POS K-map, the procedure is the same, except that we map 0's.
- Let:
$f = (\overline{w} + \overline{x} + \overline{y} + \overline{z}) \cdot (\overline{w} + \overline{x} + \overline{y} + z)$
$\cdot (\overline{w} + x + \overline{y} + \overline{z}) \cdot (\overline{w} + x + \overline{y} + z)$
- We find prime implicants exactly the same way – except that we look for variable that produce 0's.
- As shown, w and y do not change over the extent of the function.
- The simplified expression is: $f = (\overline{w} + \overline{y})$. The simplified circuit is shown at right.

	$y+z$	$y+\overline{z}$	$\overline{y}+z$	$\overline{y}+\overline{z}$
$w+x$	0000 0 0	0001 1 1	0011 3 3	0010 2 2
$w+\overline{x}$	0100 4 4	0101 5 5	0111 7 7	0110 6 6
$\overline{w}+\overline{x}$	1100 C 12	1101 D 13	1111 0 F 15	1110 0 E 14
$\overline{w}+x$	1000 8 8	1001 9 9	1011 0 B 11	1010 0 A 10



Summary of Karnaugh Map Procedure

- **In summary, to simplify a Boolean expression using a K-Map:**
  1. **Start with the truth table or Boolean expression, if you have one.**
  2. **If starting from the truth table, write the Boolean expression for each truth table term that is 1 (if SOP) or 0 (if POS).**
  3. **(Develop the full Boolean expression, if necessary, by OR-ing the AND terms together if SOP or AND-ing OR terms if POS.)**
  4. **On the K-Map, plot 1's (for SOP) or 0's (for POS).**
  5. **Group implicants together to get the largest set of prime implicants possible. <u>Prime implicants may overlap each other</u>. They will always be square or rectangular groups of cells that are powers of 2 (1, 2, 4, 8).**
  6. **The variables that make up the term(s) of the new expression will be <u>those which do not vary in value over the extent of each prime implicant</u>.**
  7. **<u>Write the Boolean expression for each prime implicant and then OR (for SOP) or AND (for POS) terms together to get the new expression</u>.**

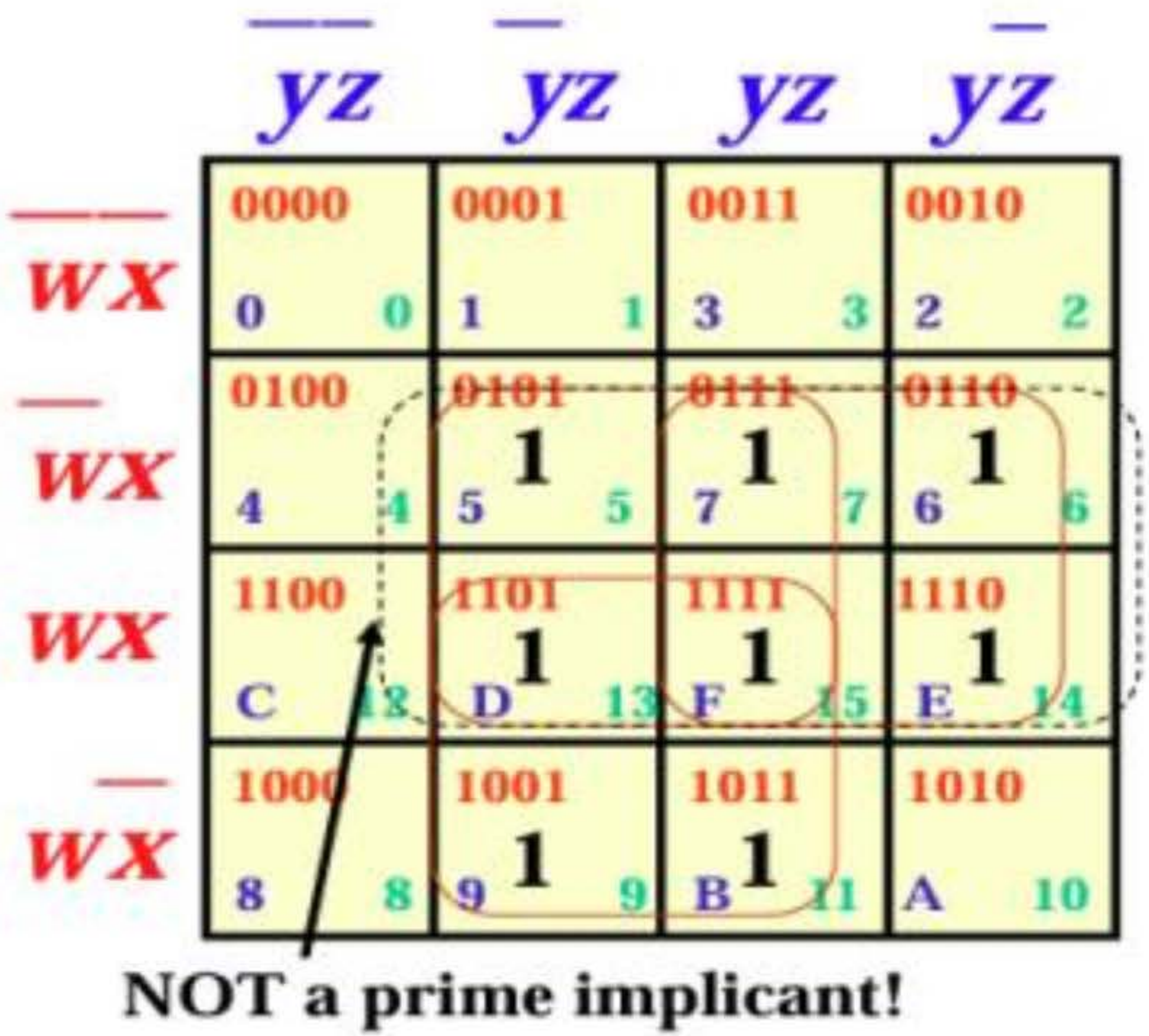
K-Map Example of Prime Implicants

- The Boolean expression represented is:

$$f = \bar{w}x\bar{y}z + \bar{w}xyz + \bar{w}xy\bar{z} + wx\bar{y}z$$
$$+ wxyz + wxy\bar{z} + w\bar{x}\bar{y}z + w\bar{x}yz$$

- Note that since prime implicants must be powers of 2, the largest group of squares we can circle is 4.
- Thus we circle three groups of 4 (in red). <u>The circles may overlap</u>.
- We now write the new SOP simplified expression. It is:

$$f = xy + xz + wz$$



Another SOP Minimization, Given the Truth Table

w	x	y	z	f
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	1
0	1	0	1	1
0	1	1	0	
0	1	1	1	
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	1
1	1	0	1	1
1	1	1	0	
1	1	1	1	

Minterms Plotted on K-Map

- The four minterms are plotted on the truth table.
- Note that they easily group into one prime implicant.
- Over the extent of the prime implicant, variables w & z vary, so they cannot be in the Boolean expression for the prime implicant.
- Variables x and y do not vary.
- Thus the expression for the minimum SOP representation must be $f = x\bar{y}$.

	$\bar{y}\bar{z}$	$\bar{y}z$	yz	$y\bar{z}$
$\bar{w}\bar{x}$	0000 0 0	0001 1 1	0011 3 3	0010 2 2
$\bar{w}x$	0100 1 4 4	0101 1 5 5	0111 7 7	0110 6 6
wx	1100 1 C 12	1101 1 D 13	1111 F 15	1110 E 14
$w\bar{x}$	1000 8 8	1001 9 9	1011 B 11	1010 A 10

The original Boolean expression is:

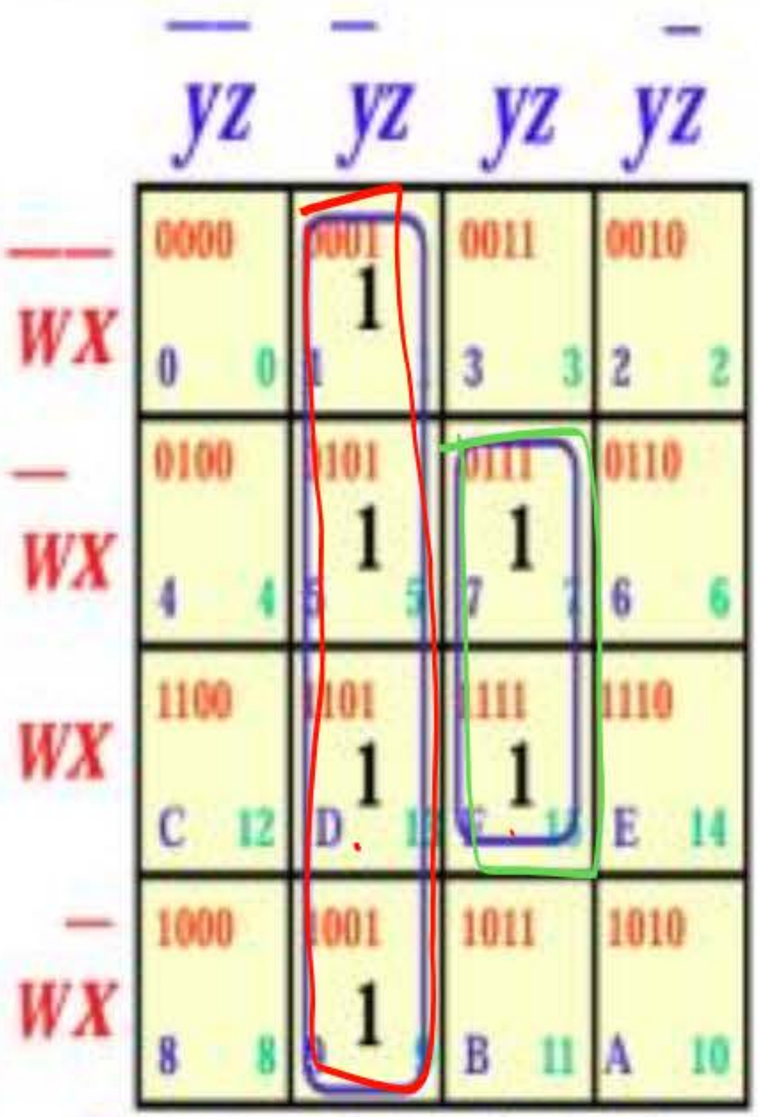
$f = \bar{w}x\bar{y}\bar{z} + \bar{w}x\bar{y}z + wx\bar{y}\bar{z} + wx\bar{y}z$

PW TEACHING WALLAH

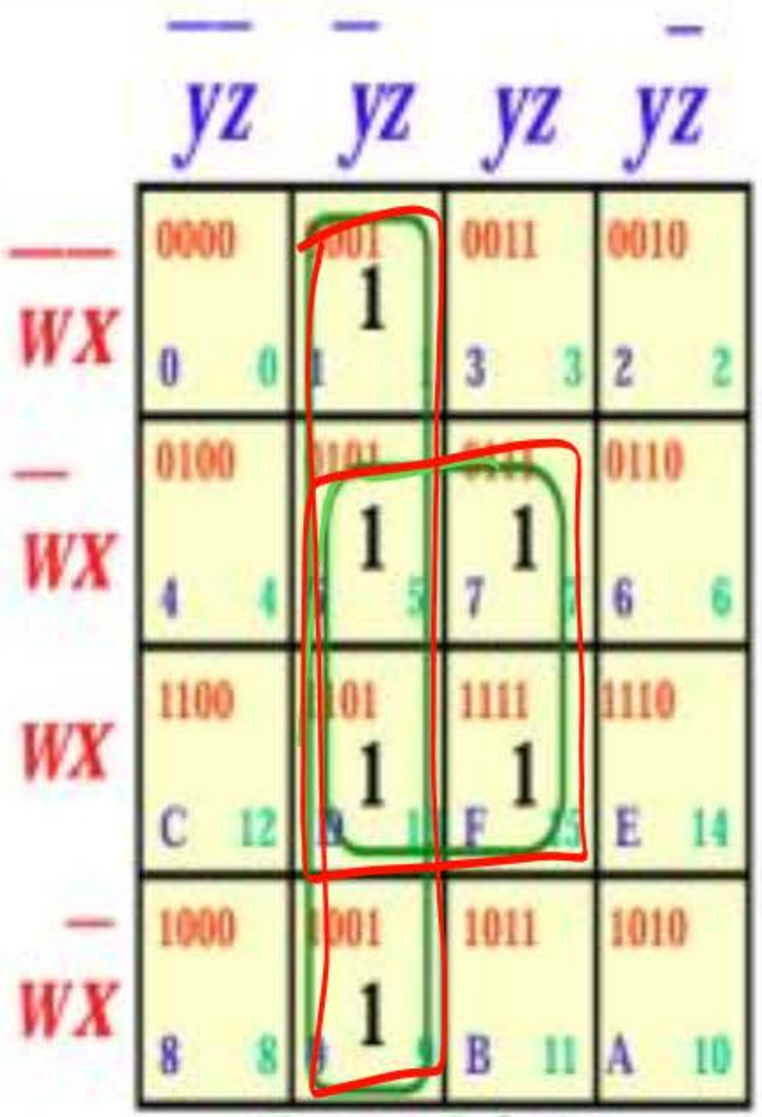
Another Example: Biggest Prime Implicants

w	x	y	z	f
0	0	0	0	
0	0	0	1	1
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	1
0	1	1	0	
0	1	1	1	1
1	0	0	0	
1	0	0	1	1
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	1
1	1	1	0	
1	1	1	1	1

Minimization Using Four-Variable K-Map



Incorrect solution (partial minimization): $f = xyz + \bar{y}z$



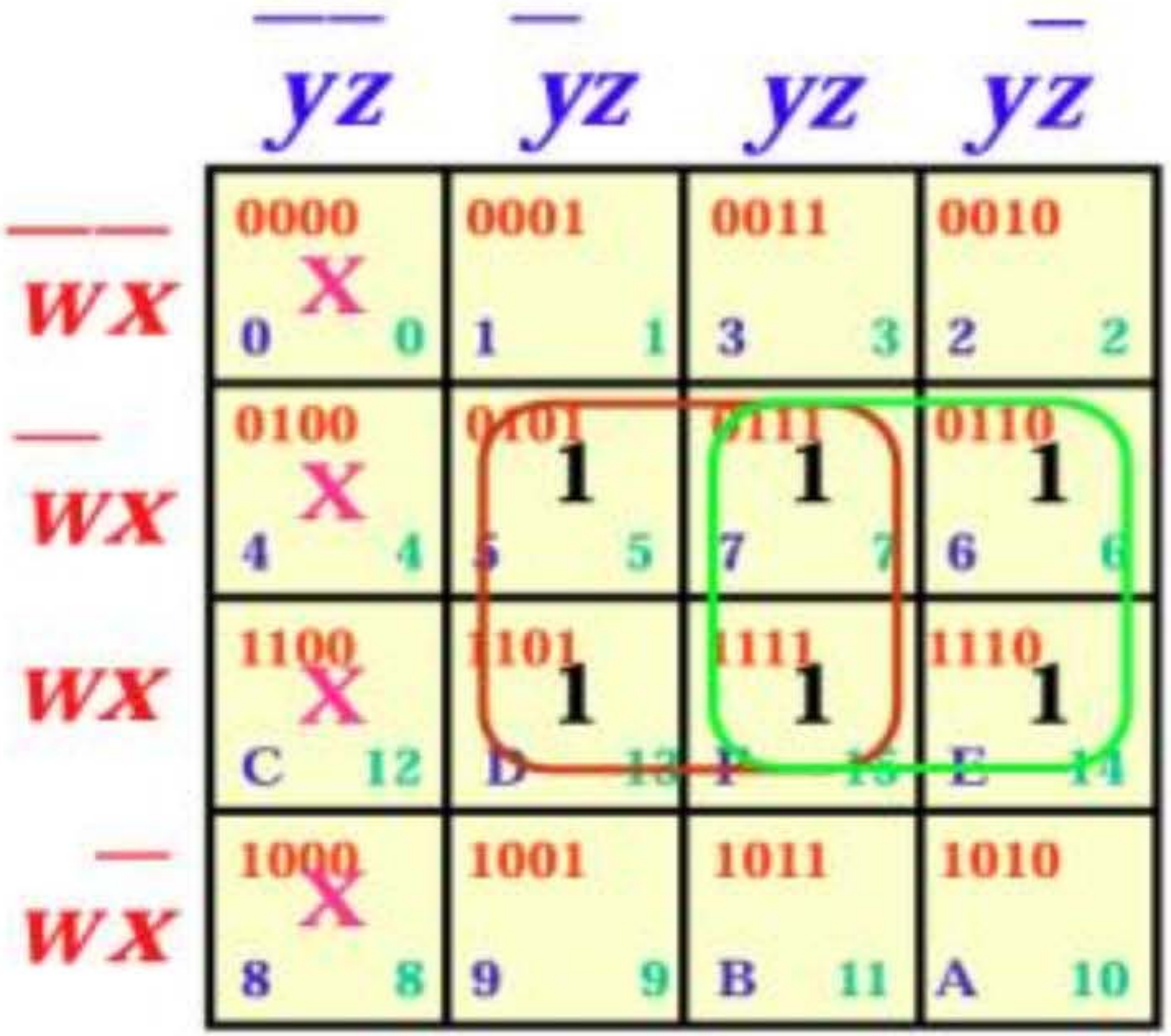
Correct Solution:

$f = xz + \bar{y}z$

Remember: Prime implicants should overlap, if this means that they can be made larger.

The Concept of "Don't Cares"

- The six implicants on the K-Map shown can be represented by the simplified expression $f = x\,y + x\,z$.
- Suppose for our particular logic system, we know that y and z will never be 0 together.
- Then the $\overline{y}\overline{z}$ implicants do not matter, since they cannot happen.
- To show the condition cannot occur, we put X's in the column -- they are "don't cares" -- conditions that cannot happen.



"Don't Cares" (2)

- **Since the function will never get into the left column, we "don't care" how those minterms are represented.** **Why not make them 1's?**
- **Doing that, we can make a much larger prime implicant and a simpler Boolean expression. For this implicant, $f = x$.**
- **The expression is valid, since the forbidden condition will never allow the two left squares to be occupied.**
- **"Don't cares" let us further simplify an expression.**

	$\overline{yz}$	$\overline{y}z$	yz	$y\overline{z}$
$\overline{wx}$	0000 0 1 0	0001 1 1	0011 3 3	0010 2 2
$\overline{w}x$	0100 4 1 4	0101 5 1 5	0111 7 1 7	0110 6 1 6
wx	1100 C 1 12	1101 D 1 13	1111 F 1 15	1110 E 1 14
$w\overline{x}$	1000 8 1 8	1001 9 9	1011 B 11	1010 A 10

"Don't Cares" – Another Example

- Assume the Boolean expression is as shown on the K-map (black 1's and black prime implicants).
- The simplest SOP expressions for the function is $f = \bar{w}xz + wxy$.
- Also assume that $wx\bar{y}\bar{z}$ and $wx\bar{y}z$ cannot occur.
- Since these cannot ever happen, they are "don't cares," and since they are "don't cares," make them 1 (red)!
- We can then further simplify the expression to $f = wx + xz$ (larger prime implicants).

	$\bar{y}\bar{z}$	$\bar{y}z$	yz	$y\bar{z}$
$\bar{w}\bar{x}$	0000 0 0	0001 1 1	0011 3 3	0010 2 2
$\bar{w}x$	0100 4 4	0101 1 5 5	0111 1 7 7	0110 6 6
wx	1100 1 C 12	1101 1 D 13	1111 1 F 15	1110 1 E 14
$w\bar{x}$	1000 8 8	1001 9 9	1011 B 11	1010 A 10

"Don't Cares" -- Summary

- "Don't Cares" occur when there are variable combinations, represented by squares on a Karnaugh map, that cannot occur in a digital circuit or Boolean expression.
- Such a square is called a "Don't Care." Since it can never happen, we can assign a value of 1 to the square and use that 1 to (possibly) construct larger prime implicants, further simplifying the circuit (you could assign it the value 0 in a POS representation).
- Circuits or Boolean expressions derived using "Don't Care" squares are as valid as any other expression or circuit.
- We will see later in sequential logic that the concept of "Don't Cares" will help in simplifying counter circuits.

A **combinational circuit**, also called a **combinational logic circuit**, is a digital electronic circuit whose output is determined by present inputs only.

The output of a combinational logic circuit does not depend on the history of the circuit operation. In other words, a combinational circuit is a digital logic circuit whose output depends only on the present input values and does not depend on any feedback or previous input or output values.

In this chapter, we will explain the fundamentals of combinational circuits, and its block diagram, types, and applications. So, lets start with the basic definition of combinational circuits.

What is a Combinational Circuit?

A combinational circuit is a type of digital logic circuit whose output depends on the present input values only and does not depend on past input and output values. Therefore, a combinational circuit is considered to not have a memory element in its circuit that stores previous inputs and outputs. Instead, it consists of a certain number of input lines to apply current input values and a certain number of output lines.

The most important characteristic of a combinational circuit is that it does not have any feedback path between input and output. Therefore, the combinational circuits can be categorized as open-loop systems.

Block Diagram of Combinational Circuit

The following figure depicts the block diagram of a combinational logic circuit.

Here, we can see that there are only three key elements in the circuit diagram of a combinational circuit, they are –

- **Input Lines** – The input lines are used to enter the input values into the combinational circuit.
- **Processing Unit** – It is the main element that processes the input values depending on the type of the circuit. For example, a full adder adds three binary bits.
- **Output Lines** – The output lines are used to take results generated by the circuit.

Characteristics of Combinational Circuits

The following are the main characteristics of combinational circuits –

- The output of a combinational circuit, at any instant of time, depends only on the present input values at that instant of time.
- Combinational circuits do not use any kind of memory element in their circuits. Thus, the previous state of input and output values do not have any effect on the present operation of the circuit.
- The output of a combinational circuit can be entirely predicted using its logical operation and input values.
- Combinational circuits produce an instantaneous output in response to any change in its input values.

Types of Combinational Circuits

In digital electronics, the combinational circuits are important components of digital systems. Depending on the functions performed, there are various types of combinational circuits. Some common types of combinational circuits and their functions are explained below –

- Binary Adders
- Binary Subtractors
- Multiplexers (MUX)
- Demultiplexers (DEMUX)
- Encoders
- Decoders
- Comparators

THANK YOU